Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Chapter 8

Diverse Problems

Dr Rahaf Ghazal

2025-2026

Longest Common Subsequence (LCS) Problem

Given two strings, s1 and s2, the task is to find the length of the Longest Common Subsequence. If there is no common subsequence, return 0. A subsequence is a string generated from the original string by deleting 0 or more characters, without changing the relative order of the remaining characters.
For example, subsequences of "ABC" are " ", "A", "B", "C", "AB", "AC", "BC" and "ABC". In general, a string of length n has 2n subsequences.

Examples:

Input: s1 = "ABC", s2 = "ACD"
Output: 2
Explanation: The longest subsequence which is present in both strings is "AC".

Input: s1 = "AGGTAB", s2 = "GXTXAYB"
Output: 4
Explanation: The longest common subsequence is "GTAB".

Applications of LCS algorithm

LCS Algorithm has several real-world use cases, some of which are as follows:

- المعلوماتية الحيوية(Bioinformatics): تُستخدم لمقارنة سلاسل الـ DNA أو البروتينات لتحديد مدى التشابه الجيني بين الكائنات الحية
- كشف السرقة الأدبية(Plagiarism Detection): تعتمد عليها البرامج المتخصصة لمقارنة الأبحاث أو المقالات مع قواعد بيانات ضخمة، حيث تحدد النصوص المتطابقة حتى لو حاول الشخص التلاعب بترتيب الجمل أو حذف كلمات بسيطة
- ضغط البيانات(Data Compression): تُستخدم في بعض خوارزميات الضغط لتحديد الأنماط المتكررة في البيانات لتقليل حجم التخزين المطلوب
- مدققات الإملاء والإكمال التلقائي: من خلال إيجاد تسلسل LCS بين كلمة بها خطأ إملائي وكلمات في القاموس، يُمكن لمدققات الإملاء اقتراح الكلمة المقصودة الأرجح.

LCS algorithm-Using recursion Approach(Brute Force Search):

الفكرة هي مقارنة آخر محرف من سلسلتين تملكان الطول (n,m)، عندئذ لدينا نتيجتين:

- إما التطابق: وهذا يمكننا من الاستدعاء العودي للدالة من أجل ماتبقى من السلسلتين (n-1,m-1)
- أو عدم التطابق : وهنا نختبر الدالة عودياً مرتين: [من أجل السلسلة الاولى كاملة مع الثانية بعد حذف آخر محرف منها (n,m-1)] - [ومن أجل السلسلة الاولى بعد حذف آخر محرف منها مع الثانية كاملة(n-1,m)].

Pseudocode:

```
int lcsRec(s1, s2, m, n) {
   if (m == 0 || n == 0)     return 0;
   if (s1[m - 1] == s2[n - 1])
      return 1 + lcsRec(s1, s2, m - 1, n - 1);
   else
      return max(lcsRec(s1, s2, m, n - 1), lcsRec(s1, s2, m - 1, n))
 }
```

Time Complexity:
$O(2^{\min(n,m)})$

ملاحظة : يمكن تعديل الخوارزمية السابقة لتقوم بطباعة السلسة الاطول وليس حساب طولها بالشكل التالي:

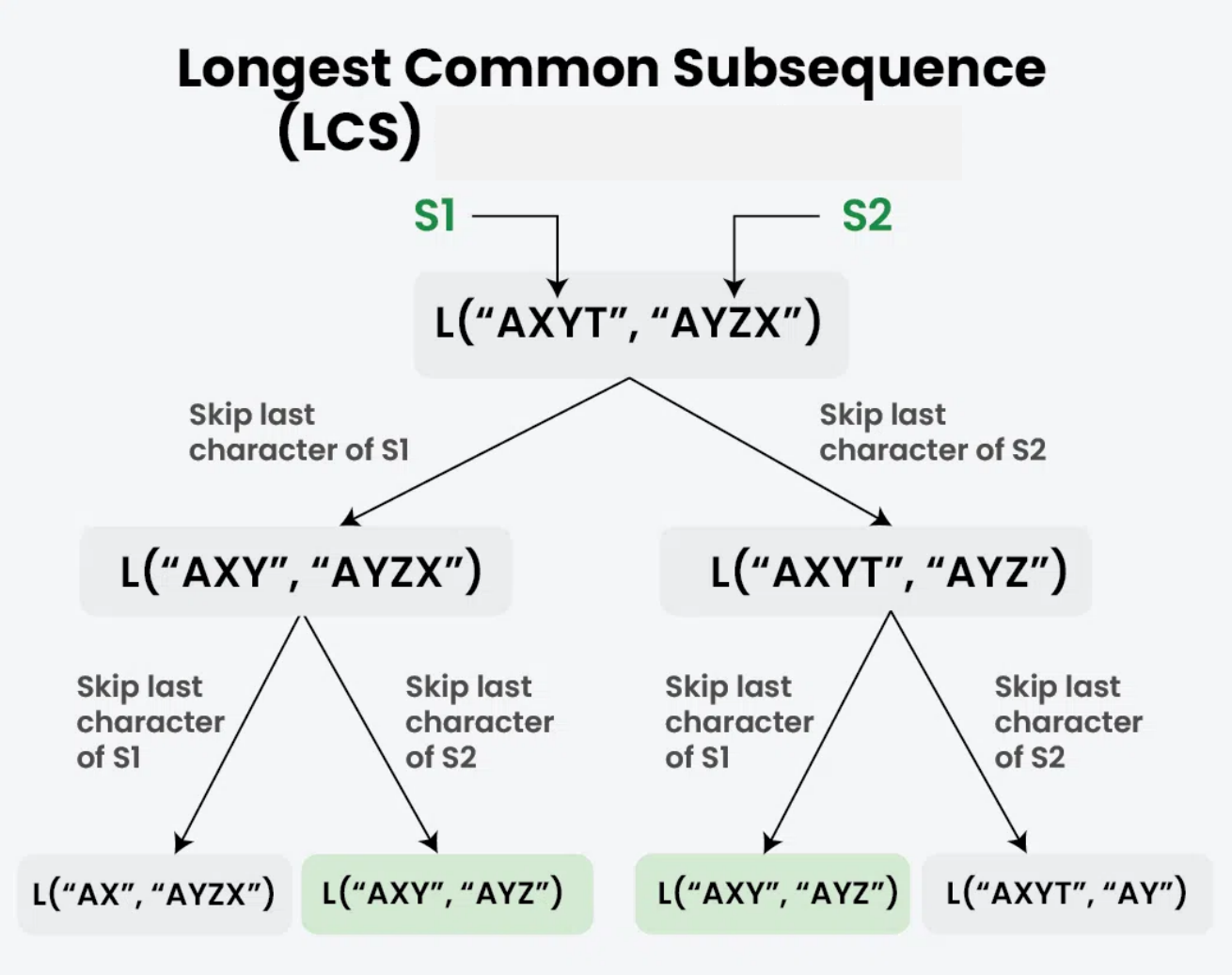
```
string getLCS(s1, s2, m, n) {
    // القاعدة الأساسية: إذا انتهت إحدى السلسلتين
    if (m == 0 || n == 0) return "";

    // حالة التطابق: نأخذ الحرف ونضيفه للنتيجة
    if (s1[m-1] == s2[n-1]) {
        return getLCS(s1, s2, m-1, n-1) + s1[m-1];
    }

    // حالة الاختلاف: نقارن المسارين ونختار السلسلة الأطول بينهما
    string res1 = getLCS(s1, s2, m-1, n);
    string res2 = getLCS(s1, s2, m, n-1);

    return (res1.length() > res2.length()) ? res1 : res2;
}
```

Oops!! There are Overlapping Subproblems in Recursion tree



LCS algorithm -Using Top-Down DP (Memoization) Approach:

To optimize the recursive solution, we use a **2D memoization table** of size mxn initialized to −1. Before making recursive calls, we check this table to avoid redundant computations of overlapping subproblems.

Pseudocode:

```
int lcsRec(s1, s2, m, n) {
if (m == 0 || n == 0)          return 0;
if (memo[m][n] != -1)          return memo[m][n];
if (s1[m - 1] == s2[n - 1])
      return memo[m][n] = 1 + lcsRec(s1, s2, m - 1, n - 1)
return memo[m][n] = max(lcsRec(s1, s2, m, n - 1), lcsRec(s1, s2, m - 1, n))
}
```

Time Complexity:
O(n x m)

Try to solve LCS problem by Bottom-Up DP (Tabulation) approach

Traveling Salesman Problem(TSP):

Given a 2d matrix **cost**[][] of dimensions **n** * **n** where **cost**[**i**][**j**] denotes the cost of moving from city **i** to city **j**. Find the minimum cost to complete a tour from city 0 (0-based index) to all other cities such that we visit each city exactly once and then at the end come back to city 0[**is named Hamiltonian circuit in graph theory**].

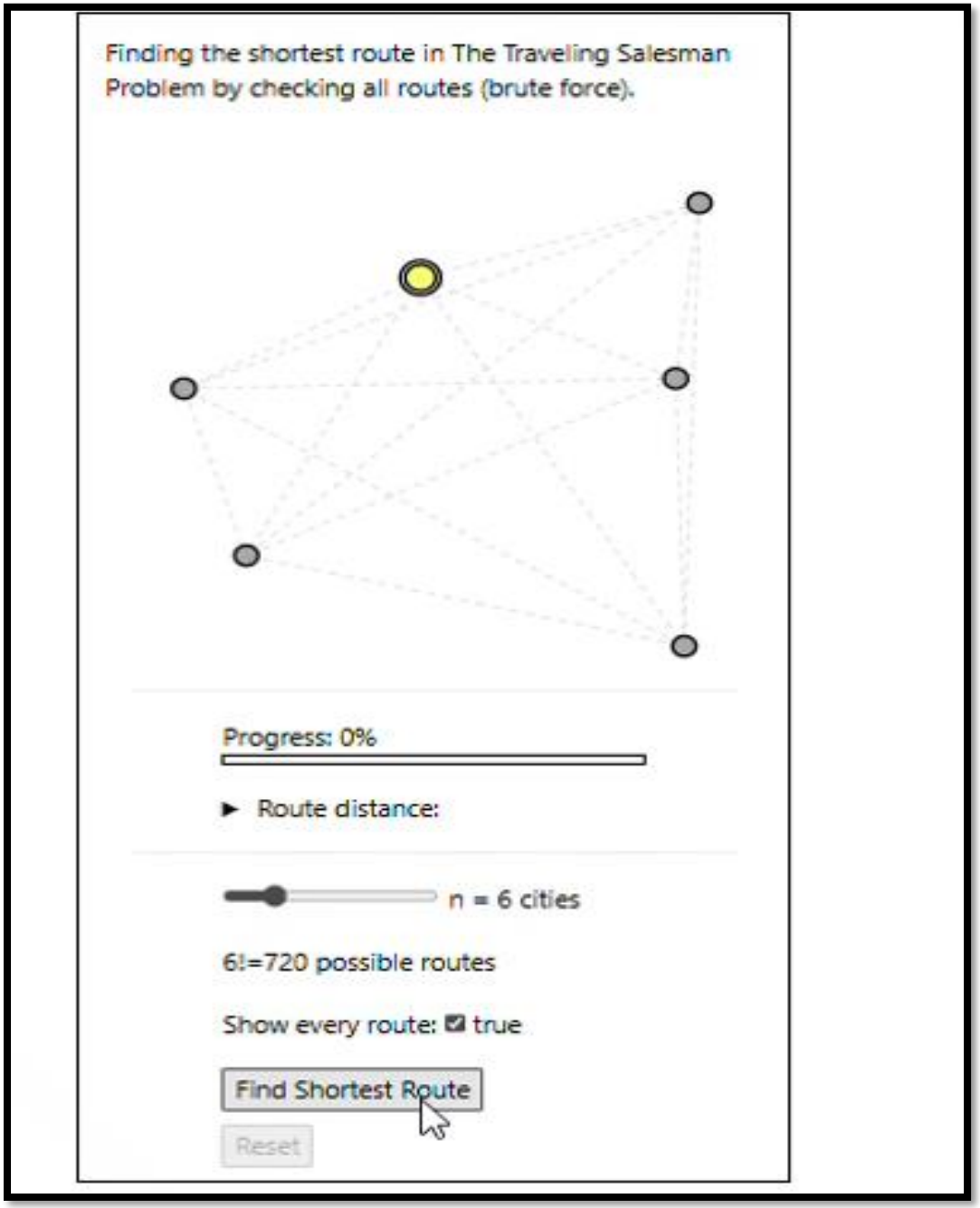
Example:

Input: cost[][] = [[0, 10, 15, 20],
[10, 0, 35, 25],
[15, 35, 0, 30],
[20, 25, 30, 0]]

Output: 80

Explanation: We can visit 0->2->3->1->0 and cost = 15 + 30 + 25+ 10= 80.

TSP -Using recursion Approach(Brute Force Search):



الفكرة هي: لإيجاد الحل الأمثل لمسألة TS، سنقوم بفحص جميع الطرق الممكنة، وفي كل مرة نجد طريقًا أقصر، سنقوم بتخزينه، بحيث نحصل في النهاية على أقصر طريق.

Time Complexity: O(n!)

TSP -Using Memoization (Top Down DP) Approach:

الفكرة هي: استخدام الخوارزمية العودية السابقة ولكن مع الاحتفاظ بنتائج عمليات البحث السابقة لمنع التكرار في الحسابات . ولكن بالمقابل سوف نحصل على خوارزمية بتعقيد زمني قدره $O(2^n * n)$

making this approach much faster than O(n!) but still exponential

Dijkstra's shortest path algorithm

Dijkstra's algorithm finds the shortest path from a single source node to all other nodes in a weighted graph with non-negative edge weights. It uses a greedy approach, iteratively selecting the unvisited node with the smallest tentative distance, updating its neighbors, and marking it visited, typically optimized using a priority queue.

Applications of Dijkstra's shortest path algorithm

- **Network Routing**: Finding the shortest path in computer networks
- **GPS Systems:** Calculating the fastest driving route between locations
- **Map Applications:** Determining the shortest path in transportation networks.

Pseudocode:

```
Dijkstra(Graph, start):

dist[all] = ∞, dist[start] = 0

while(unvisited_nodes exist):

    u = unvisited_node_with_min_dist

    for neighbor v of u:

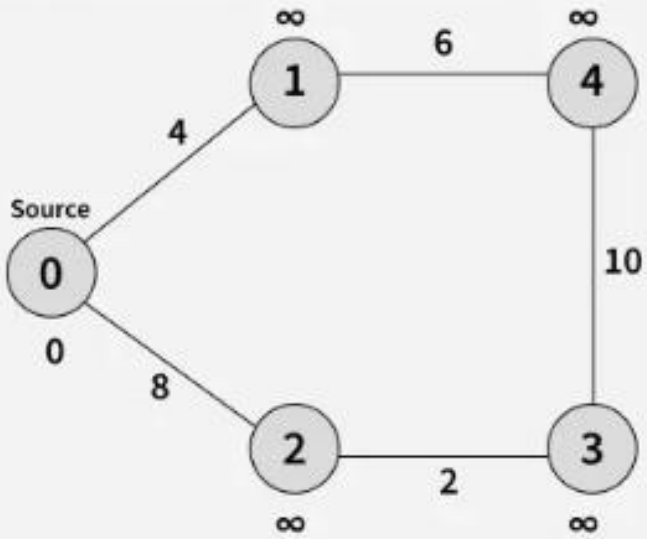
        if (dist[u] + weight(u,v) < dist[v]):

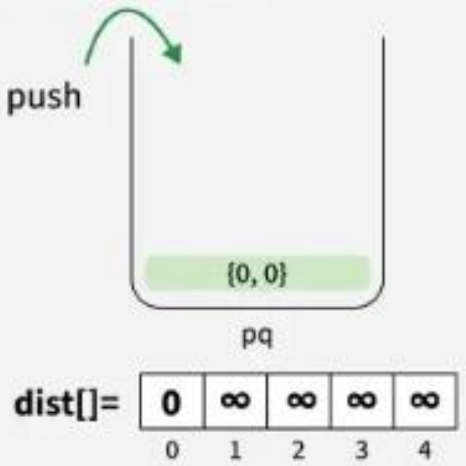
        dist[v] = dist[u] + weight(u,v)

    mark u as visited
```

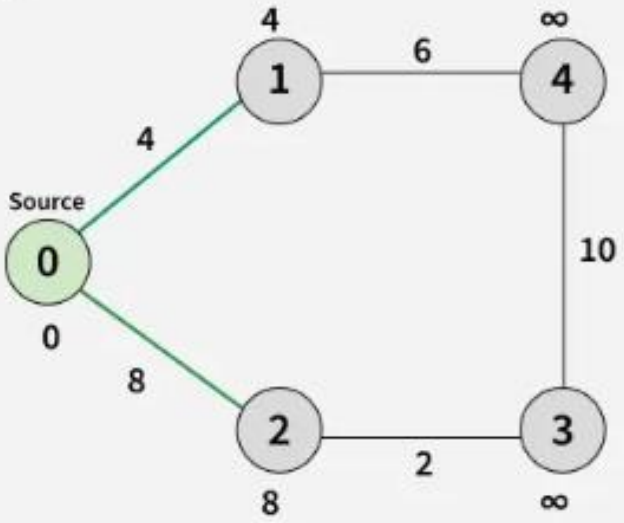
Example

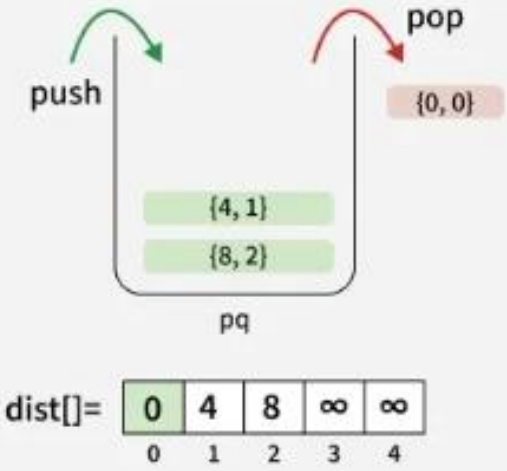
01 Step Maintain a dist[] array initialized with 0 for the source node and ∞ for all others. Use a priority queue pq to store {weight, node} pairs for selecting the minimum unprocessed node, and insert {0, source} into the pq to start from the source node.





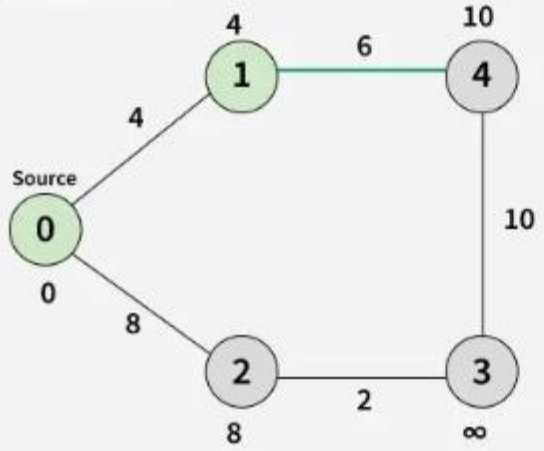
02 Step Pop the first element from the pq (e.g., {0, 0} where u = 0). For each adjacent node v of u, if dist[v] > dist[u] + weight, update dist[v] to dist[u] + weight and insert {dist[v], v} into the pq.

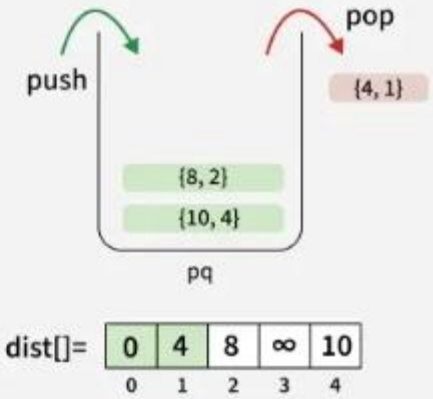




03 Step

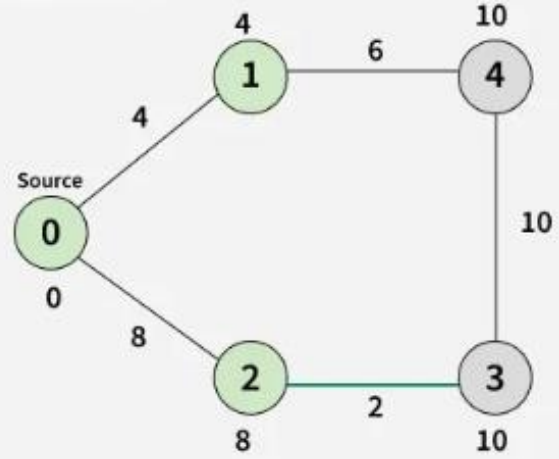
Pop the first element from the pq (e.g., {4, 1} where u = 1). For each adjacent node v of u, if dist[v] > dist[u] + weight, update dist[v] to dist[u] + weight and insert {dist[v], v} into the pq.

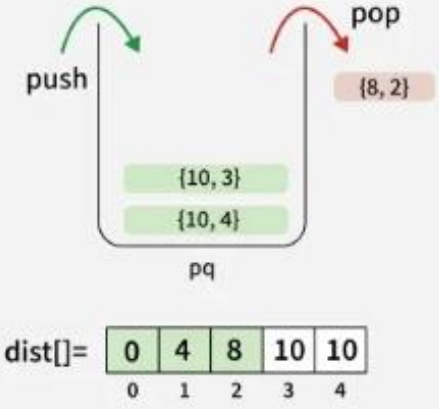




04 Step

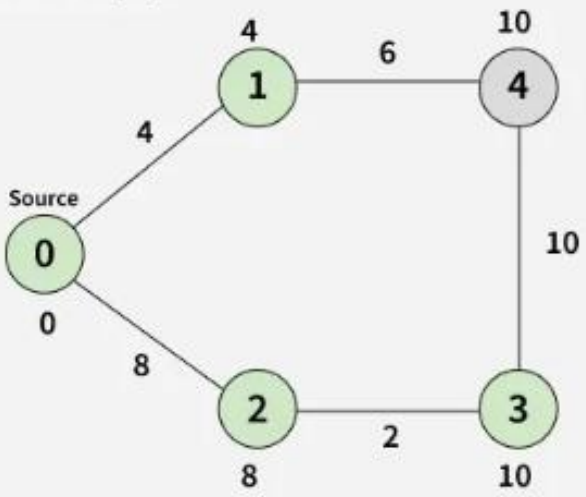
Pop the first element from the pq (e.g., {8, 2} where u = 2). For each adjacent node v of u, if dist[v] > dist[u] + weight, update dist[v] to dist[u] + weight and insert {dist[v], v} into the pq

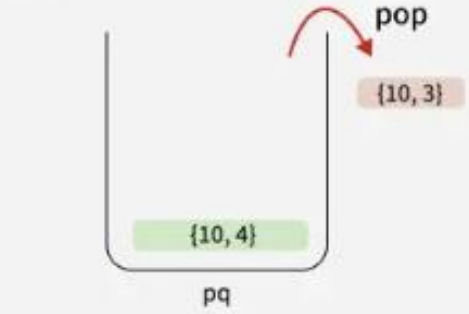




05 Step

Pop the first element from the set (e.g., {10, 3} where u = 3). For each adjacent node v of u, if dist[v] > dist[u] + weight, update dist[v] to dist[u] + weight and insert {dist[v], v} into the pop



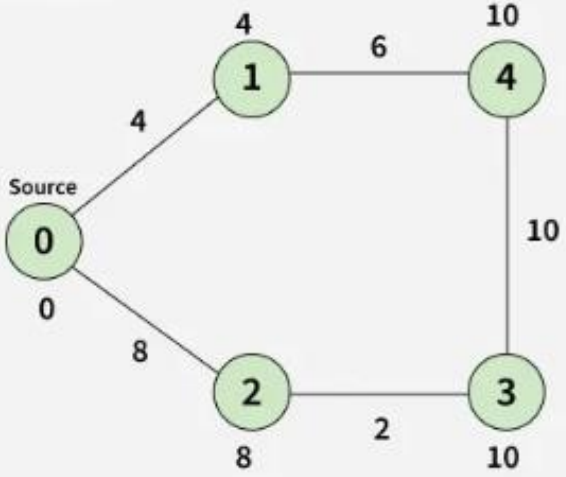


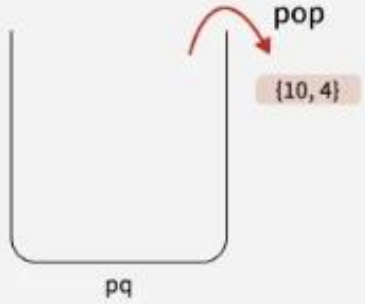
dist[]=

0	4	8	10	10
0	1	2	3	4

06 Step

Pop the first element from the pq (e.g., {10, 4} where u = 4). For each adjacent node v of u, if dist[v] > dist[u] + weight, update dist[v] to dist[u] + weight and insert {dist[v], v} into the pq



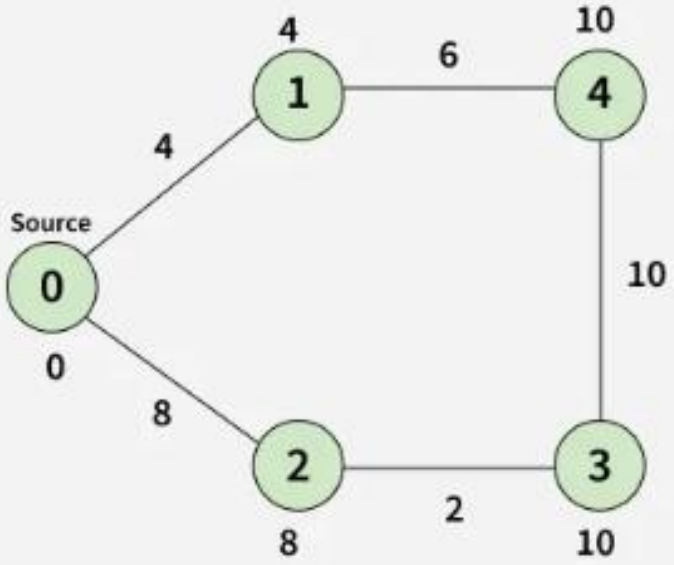


dist[]=

0	4	8	10	10
0	1	2	3	4

07 Step

All nodes are processed and the pq is empty,
We now have the final shortest distances from the source node 0.





dist[]=

0	4	8	10	10
0	1	2	3	4

The End

